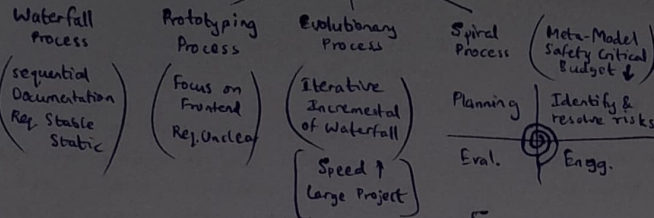
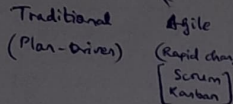


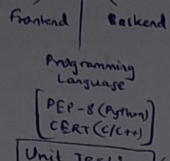
Traditional Models



Models



(Smells: Tech. debt, Code)



Exploratory Programming:
Build and Fix
Quick Development
Small & Large x
Knight Capital Group
Financial critical system

Req.: Statement that specifies what a system must do or qualities it must possess to satisfy what stakeholder needs.

Verification: Am I building the product right (Analysts)

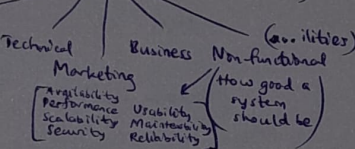
Validation: Am I building the right product (Stakeholders)

Scope creep → Not clear functionalities

Acceptance Testing (15%)
(Version control, Tracking)

(Elicitation, Analysis, Negotiation, Doc., val.)
Req. Dev.

Requirements Engg. (PRD → SRS)



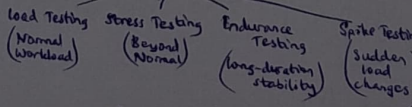
PDCA → PDSA
Check (Testing & Deployment)
Do (Implementation)
Plan (Planning, Estimating, Req., Design)
Act (Study (Retrospect))

Interview
Observation
JAD
Prototyping (Paper → Figure)

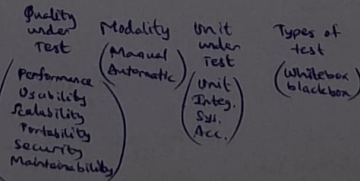
Decision Table: (Black Box)

Condition	Rule 1	Rule 2
Yes/No	Yes/No	Yes/No
1	x	y
2		
3		
4		
5		

Performance Testing (Response Time, throughput)



Dimensions of Testing



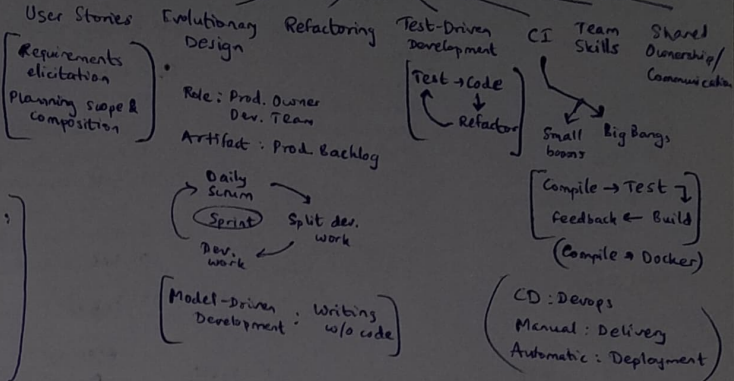
Security Testing: Penetration Testing, Fuzz Testing

Usability Testing: Task completion rate, User satisfaction

Scenario: Basic + Alternate + combination of flows

Dataflow Testing
LHS → Defining Node (IN)
RHS → Usage Node (OUT)

Agile Models



Designing
Dev. [SW]
CI

Manage OS
Database

Operations
Monitor load spikes
Performance crashes
Backups
Rollback releases
Hardware updates

MoSCoW
Must
Should
Could
Won't

KANO
Basic (Differentiators)
Delighters

Design
High-level
Software Architecture (Artifact, design, doc.)
System Testing (20%)

Low-level
Detailed Design (Interface, Document & UML)
Integration Testing (25%)

Conformance Testing

Static Testing
(Inspection)
Prevent defects
Analyse code
Req. doc.
defect early (find)

Dynamic Testing
(Unit, Integ., Syst.)
Detect defects
Code
System/requirements

Req./Planning: 30-40%
Proj. Mgmt/Verif.: 20-30%
People & Org.: 15-25%
Tech./Design: 10-20%
Testing/QA: 5-15%
Change Mgmt: 5-15%

Bad. Req.:
Inconsistent - Contradict
Incomplete - Omitted
No numbers - Approx.

Unit-Testing stubbing:

Understand Req.
Complex logic
Dependencies (stub)
State mgmt.
Test maintainability
Performance overhead
FP/FN
Test coverage
Testing edgecases
Test independence

System Testing:
Real world env.
Speed, Latency, Load
User & designer
Last place of fault

Integration Testing:
correctly interact with each other
Big-bang
Bottom-up
Top-down
Critical first

Acceptance Testing:
crowd compliance
Field
Business req.
User workflow
Real-world use
Regulatory rules

Alpha Testing (supervision)
Beta Testing (Normal env.)

White-Box Testing
(All levels - code)

Glass-Box, Structured

Coverage
Statement
Branch
Decision
Condition

Dataflow Mutation

Path predicate

Combinatorial

Error guessing

Model Based

Static program analysis

Black-Box Testing - System level
(Requirements - SRS)

(i) **Equivalence Partitioning** - Divide into valid/invalid classes

(ii) **Boundary value Analysis** - Edge values

(iii) **Decision Table** - Complex business rules

(iv) **State Transition** - val. behaviour across states

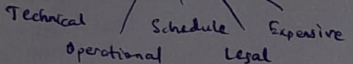
Equivalence groups treated same by algorithm

Regression functions work after changes

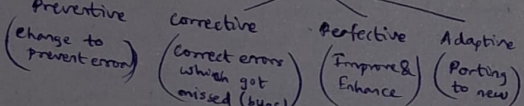
SDLC (Deming's Cycle):

- 1) Feasibility Study - Planning, Tracking, Estimation, Artifacts
- 2) Req. Analysis & Specs - Documentation
- 3) Design
- 4) Implementation / coding
- 5) Testing & Deployment (Ops) - Exhaustive Testing, Random Testing
- 6) Maintenance

Feasibility Study



Maintenance

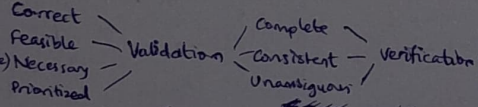


[Bug: When Req. don't work]

Severity: Impact on System

Priority: Urgency for fixing

(Blocker, Critical, Major, minor, Trivial)



External Inputs: Input CU data (CUD) (3, 4, 6)

External Outputs: Outputs sent out (4, 5, 7)

External Inquiries: queries that retrieve data with less processing (3, 4, 6)

Internal logic files: App-maintained logical data groups (7, 10, 15)

External Interface files: Read only logical data used by app maintained externally (5, 7, 10)

VAF = 1.20

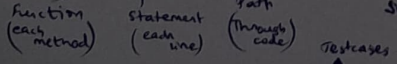
FPA: 143 FP → 2860 hr/20 hr per FP

5 days → 150 hr/week

Timeline → 2860/150 = 19 weeks

Buffer @ 20% → 23 weeks

Coverage



Needs → Features → Usecases → Scenario
Testing process: Planning → Design → Execution → Evaluation → Closure

Software Design

Abstraction
Separation of concerns → behavior as part of req.
Modularity → Monolithic x
Information hiding → Encapsulation
Refinement
Functional Independence
Cohesion & Coupling

Coupling: Interdependence
↳ one module and other

Cohesion: one module - one function

S → Single responsibility
O → Open/closed principle
L → Liskov's substitution
I → Interface segregation
D → Dependency Inversion

GRASP

Creator
Information Expert
Loose Coupling
High Cohesion
Controller
Polymorphism
Pure Fabrication
Indirection
Protected Vars

Lehman & Ledley (1974)
Continuing change
Increasing complexity
Code Review: Sonar/Pube

Responsibility
Doing
Knowing

Mental → Jakob's Model
[Skeumorphism]

Inside-out: system → Interface
Outside-In: Interface → system

Usability:

Extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in specified context of use.

(User-driven)

Learnability
Efficiency
Memorability
Errors
Satisfaction

(if-else)

alt → alternate
opt → optional
break → Break
loop → loop
seq → sequential
strict → strict
par → parallel
critical → critical
ignore → Ignore
consider → consider
neg → Negative
assert → Assert

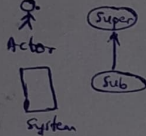
Modeling Languages

Domain-specific languages
(HTML, SPL)

General Purpose Modeling Langs.
(UML, XML)

SRS: (Upon Where it-so that)

Field
System ID
Title
Statement
Preconditions
Postconditions
Priority
Acc. Criteria
Notes



UML: → Conceptual classes (Nouns)

No.
Name
Overview
Type
Actors
Properties
Constraints
Precondition
Flow (Main & Alt)
Postcondition
Cross ref.

Attributes (Access Modifiers)
↓
Multipliers
↓
UML

F Types
Transactional PT
Input Output Inquiries
Data PT
logical External files
Internal Interface files

- count no. of Fns. per category
- Complexity factor
- VAF Multiplier
- FP Total
- Calc. Time

I. Aggr. ind. est.
II. Range, Not No.
III. Why estimate
IV. Est. ≠ Commitment
V. Measure count, not estimate.

Req. Gathering:

SRS + Type + Source
(P/N/E) (stakeholders)

Business Needs ↔ Req. System design

MDC - Safety Critical (Modified Condition Decision)

Error: Human Mistake

Fault: Defect (Flaw)

Failure: system Dev

ISQ8

(1) Testing shows presence of defects

(2) Exhaustive Testing: Impossible (risk-based selection)

(3) Early testing saves cost and time

(4) Defects clusters in few modules

(5) Pesticide Paradox - Op grade test → New bugs

(6) Testing depends on context (web app vs. safety critical)

(7) Absence of errors

Fixing defects ≠ success

Earned value management:

Planned Value
Earned Value (CPI, SPI)
Actual Cost
↓
EV AC EV PV

CPI > 1: Under Budget

CPI < 1: Over Budget

SPI > 1: Ahead of Schedule

SPI < 1: Behind Schedule

EAC = $\frac{BAC}{CPI}$

ETC = EAC - AC

VAC = BAC - EAC

CV = EV - AC

SV = EV - PV

State transition table:

Current state	Event	Action	Next state

Path testing:

V(G): no. of paths

McCabe's cyclomatic Complexity: $\text{complexity} = \text{no. of simple decisions} + 1$

no. of enclosed areas + 1

Decisions → Binary

[C = edges - nodes + 2]

Regression: Loss of functionality
↓
repair of other/introduce new features

Testing ⇒ Cert. [Defects assess safety]

Wideband Delphi - bias? [Group Consensus]

Scheduling

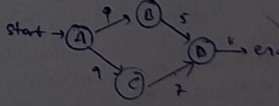
Gantt Chart
Easy but Vague
[No interdependency]
Network Tech.
CPM (Critical Path Method)
PERT (Program eval. & Review Tech.)

Scheduling

What-to-do WBS (Work Breakdown Structure)
How much (Est.)
Dependency Graph

WBS

Process [Req. Analysis, Design, Testing] (PM)
Product [Interface, DB] (Buss. Mgr.)
Hybrid



Critical path: longest in precedence network
stack: LS-ES
Time: LF-EF
↓
earliest
Max. allowable delay for a non-essential activity

Kanban board
- Backlog
- In progress
- Done

GitHub
- Branching
- Merging & Conflict mgmt
- History